```
bin/rails data_generators:generate_all
```

After running the generators above you should have at least 20,000 users.

To copy data rows, if you're unfamiliar with this technique review the SQL below. You may wish to run this and then TRUNCATE the users_copy table once you've done that, in order to test out the technique; or, you may skip running this if you're familiar with the INSERT INTO ... SELECT * FROM format being used.

The input to the INSERT statement is the output of the SELECT statement. Since the destination side table has all the database objects including indexes, constraints, defaults, triggers, and more, the insert process is slower than it would be without all of those objects. This will be significant later on!

The general pattern used for Table Copying is below. Running this is optional.

sql/table_copying_insert_into.sql

```
INSERT INTO users_copy(
  first_name,
  last_name,
  email,
  type,
  created_at,
  updated_at
)
(
  SELECT
    first_name,
    last_name,
    email,
    type,
    created_at,
    updated_at
  FROM users
);
```

If you ran the statement above in psql, you should see output something like INSERT 0 20300 indicating that around 20,000 rows were inserted.

You've now copied rows from one table to another. Rows were copied over as-is though, and no text was scrubbed.

Now you're ready to fold the scrubbing into your recipe.

As you did earlier, you'll call the scrubber functions using a SELECT statement. You'll replace the places where the table field was selected directly and instead you'll pass it through the function as input.

It's time to give that a try! Since you want to keep the users_copy table but remove the rows, from psql run TRUNCATE users_copy;.